



ENSIIE - HPC/BigData (Programmation Parallèle) Durée : 1 heure 45 mins

julien.jaeger@cea.fr, bouhrour.stephane@gmail.com

Toutes les réponses doivent être justifiées.
 La qualité de la rédaction est prise en compte.

I Questions de cours

Q.1: Donnez le nom des opérations collectives non-bloquantes ci-dessous. Les données sont à l'horizontal, les tâches MPI sont en vertical.

1

Entrée		
A_0	A_1	A_2

Sortie		
A_0		
A_1		
A_2		

2

Entrée		
A_0	A_1	A_2
B_0	B_1	B_2
C_0	C_1	C_2

Sortie		
A_0	B_0	C_0
A_1	B_1	C_1
A_2	B_2	C_2

3

Entrée		
A_0	A_1	A_2
B_0	B_1	B_2
C_0	C_1	C_2

Sortie		
A_0	B_1	C_2
A_1	B_2	C_0
A_2	B_0	C_1

Q.2: Quel est l'intérêt des appels MPI non-bloquants ? Que veut-on dire par "recouvrement des communications" ?

Q.3: Quelles sont les contraintes d'utilisation des collectives non-bloquantes ?

Q.4: Quelles sont les différentes façons de faire progresser des communications asynchrones ? Quels sont leurs avantages et inconvénients ?

II Echanges de mailles fantômes (1)

Pour cette partie, nous allons considérer un code de stencil.

Pour rappel un stencil est un code opérant sur une grille cartésienne. La mise à jour d'une case de cette grille nécessite le concours de case voisine. Dans notre cas, nous allons considérer une grille de 2D. Pour mettre à jour une case, nous avons besoin des 8 cases voisines. En bord de grille, nous considérons que les cases inexistantes apportent une contribution de valeur 0 lors de la mise à jour de la case.

Pour être plus efficaces, nous décidons de faire ces mises à jour en parallèle sur plusieurs processus MPI. Les cases de la grille sont réparties sur H processus en hauteur et L processus en largeur, pour un total de $H \times L$ processus (avec $H > 1$ et $L > 1$).

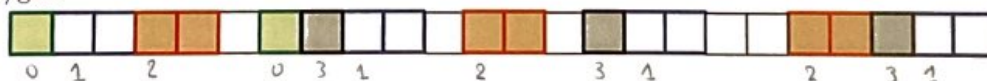
Q.5: Dans le cas général (pas de conditions aux bords), donner le rang de chaque voisin à partir du rang de la case courante (*mpirank*), de H et de L .

Q.6: Pour chaque direction, quelles sont les conditions à vérifier pour s'assurer de la présence d'un voisin ?

Q.7: A chaque itération, il est nécessaire de récupérer les cases au bord de chaque domaine voisin pour calculer la nouvelle mise à jour. En vous servant des questions précédentes, réaliser un code copiant les données à envoyer dans un buffer temporaire pour chacun des voisins, et réalisant les échanges de mailles fantômes entre les voisins.

III MPI I/O et views

Q.8: Donner et décrire les appels MPI I/O nécessaires, pour un programme s'exécutant avec 4 processus MPI, permettant de créer un nouveau fichier contenant le rang des processus selon le schéma ci-dessous (ouverture de fichier, création des view et écriture dans le fichier). Processus 0 : vert. Processus 1 : bleu. Processus 2 : rouge. Processus 3 : noir/gris.



IV Echanges de mailles fantômes (2)

Pour cette partie, nous considérons la même configuration que dans la section II.

Q.9: Donner les fonctions (et leurs paramètres) permettant de construire la topologie cartésienne décrite précédemment.

Q.10: Réaliser les échanges de mailles fantômes dans le stencil avec des collectives neighborhood. Vous pourrez vous inspirer du travail réalisé en Question 7.